Contents

Domain model – practical guide 1
1. Two-minute mental model . . . 1
2. Running example (keep these tokens) . . . 2
3. Cheat sheet – PhaseContext fields . . . 2
4. Scenario A – Start the project phase FSM (business + API) . . . 3
   Business story . . . 3
   Developer: what must be true after start in FAT (example) . . . 3
5. Scenario B – Dossier: missing → present (still not verified) . . . 3
   Business story . . . 3
   Developer: graph writes . . . 3
6. Scenario C – Verify a system requirement (test stand-in) . . . 4
   Business story . . . 4
   Before / after PhaseContext . . . 4
   Developer: preferred write path (to-be domain layer) . . . 4
   Developer: relation sketch (as-is pair) . . . 4
7. Scenario D – Document review (File is assessment subject) . . . 5
   Business story . . . 5
   Contrast with Scenario B . . . 5
8. Scenario E – Fail, re-run, then go / no-go . . . 5
   Fail then re-pass . . . 5
   Gate . . . 5
   Business view of go . . . 5
9. Role playbook (business) . . . 6
10. Developer cookbook . . . 6
   Always . . . 6
   Never . . . 6
   Mapping UI action → operation (short list) . . . 6
   As-is vs to-be (honest) . . . 7

Domain model – practical guide

Companion to the formal Domain Model. This page is a worked example: same rules, fewer class diagrams, more "what do I click / what gets written."

Audience	Use this page to…
Business (PM, V&V, dossier)	See how phase, dossier completeness, and verification show up in day-to-day work
Developers	See which nodes/relations/side effects a UI action must produce (and what never to expose as editable fields)

Formal invariants, class diagrams, and the full operation backlog stay in domain-model.md. Epic requirements: Epics.

---

1. Two-minute mental model

```
Project (PR)                = container + authority of the phase FSM
ProjectPhase (enabled set)  = allowed FSM states (VO / DO / … / FAT / …)
ProjectPhase.isActive       = current FSM state (exactly one after start)
Object (OBJ)                = product / SOI node - follower of the project FSM, not the authority
PhaseContext on items       = where that item sits relative to the project phase FSM
```

```
File Structure (dossier)     = expected document slots (presence / P5)
Requirement / File           = things we may need to verify
VerificationItem (or TestCase stand-in) = planned check + execution
```

Phase lifecycle = a finite state machine on the Project (states = enabled phases; transitions = go/no-go). Protocol = process badge on the same phase (P1 design, P2 integration, P3 onboarding, P4 escalation, P5 dossier) — not a second FSM.

Present ≠ covered ≠ verified

Word	Means	Example
Present	Slot in dossier profile has a File linked	SRS uploaded into the “SRS” folder
Covered	A check exists that assesses this subject in phase P	Test case linked to REQ as assessment subject
Verified	A successful execution against that subject as assessment subject	Test passed → REQ `verified_in` includes FAT (= project active state)

Uploading a file never verifies it. Completeness (P5) never writes `verified_in`.

---

2. Running example (keep these tokens)

Imagine project PR2026-001 “Bridge opening system”, product object OBJ2026-001-001.

Token	Type	Name
`PR2026-001`	Project	Bridge opening system (owns the phase FSM)
`OBJ2026-001-001`	Object	Bridge product (optional PhaseContext follower)
`REQ2026-001-042`	Requirement	Opening time ≤ 90 s
`FIL2026-001-010`	File	SRS-v3.pdf (deliverable)
`FIL2026-001-099`	File	fat-log-042.csv (evidence)
`TCAS2026-001-023`	TestCase	TC-FAT-023 (stand-in VI)
`TCEX2026-001-023-1`	TestCaseExecution	First run of TC-FAT-023
`STR2026-001-FILE`	Structure	Dossier profile (`meta.type: file`)

Active FSM state for the story: FAT (call it `P`). Next enabled state: SAT (call it `P'`).

UI badges you want people to recognise: FAT · P2 (phase · protocol).

---

3. Cheat sheet – PhaseContext fields

Shown on Requirement, File, TestCase / VerificationItem, … — never edited by hand.

Field	Plain language
`current`	Phase we are working this item in
`verified_in`	Phases where this item was successfully verified as assessment subject
`next required`	Phase that still needs work (or empty when done)
`protocol`	Process path badge

Project has no PhaseContext node. The phase FSM is the enabled `ProjectPhase` set plus exactly one `isActive`. On verify, P = that active project phase (not "the Object's phase").

To be determined: Phasecontext on Object.

---

4. Scenario A – Start the project phase FSM (business + API)

Business story

1. Enable phases on the project: VO, DO, UO, FAT, SAT, SIT (allowed FSM states).
2. Start the phase FSM → first enabled phase becomes active.
3. Cockpit shows one active project phase only — shared by all Objects in the project.

Developer: what must be true after start in FAT (example)

```
ProjectPhase FAT.isActive = true          (exactly one active - FSM current state)
Requirement REQ… PhaseContext (optional follower):
  current        = FAT
  verified_in    = empty
  next required  = SAT (or empty if last)
  protocol       = default for stage (P2 in integration)
Project PR…     → no PhaseContext; authority = ProjectPhase.isActive
```

Reject a second active phase. Reject activating a phase that was not enabled.

Related requirements: Epic 1 – E1-R01…R06 / user stories.

---

5. Scenario B – Dossier: missing → present (still not verified)

Business story

1. Onboarding copies the template dossier tree into the project (protocol P3).
2. Slot `1.2` "SRS" is mandatory for FAT, empty → missing.
3. Dossier manager uploads `SRS-v3.pdf` and links it into slot `1.2` → present.
4. Completeness check (P5): that slot is OK for presence.
5. The SRS is not approved yet — `verified_in` on the File stays empty until a document-review check passes.

Developer: graph writes

```
# Profile tree (as-is Structure)
PR --HAS_STRUCTURE--> STR2026-001-FILE   (meta.type = file)
STR --HAS_CHILD--> structure_child child="1.2"

# Link file into slot (presence only)
structure_child --HAS_LINK--> FIL2026-001-010

# Side effects
slot 1.2                          → present
FIL2026-001-010.verified_in       → UNCHANGED (still empty)
no VerificationItem created from presence alone
ReadinessCheck.dossierComplete    → may become true if all mandatory slots for FAT are present
protocol on completeness action   → P5
```

Rule of thumb for QA: if completeness turns green but File still has empty `verified_in`, that is correct.

---

6. Scenario C – Verify a system requirement (test stand-in)

Until Epic 4 ships, TestCase + TestCaseExecution stand in for a test-kind VerificationItem.

Business story

1. Create requirement `REQ2026-001-042` “Opening time ≤ 90 s”.
2. Create test case `TCAS2026-001-023` and link it so the REQ is the assessment subject.
3. Run the test → pass; attach log `FIL2026-001-099` as evidence.
4. Requirement is verified in FAT (project active FSM state); the log file is not verified by being evidence.

Before / after PhaseContext

After create requirement (in FAT)

Item	current	verified_in	next required
REQ…042	FAT	empty	FAT

After create + link test (covered, not verified)

Item	current	verified_in	next required
TCAS…023	FAT	empty	FAT
REQ…042	FAT	empty	FAT (unchanged — only covered)

After pass execution

Item	current	verified_in	next required
TCAS…023	FAT	{FAT}	empty
TCEX…1	FAT	{FAT}	n/a
REQ…042	FAT	{FAT}	SAT (or empty)
FIL…099 (evidence)	—	unchanged	—

Developer: preferred write path (to-be domain layer)

Do not recreate today’s FE pattern (orphan test case in Items, then Linker). Prefer one command:

```
CreateTestCaseForRequirement(project, REQ…042, name=TC-FAT-023)
  → creates TCAS + bidirectional IS_* links + PhaseContext side effects
ExecuteTestCase(TCAS…023, result=pass, evidence=[FIL…099])
  → creates TCEX, stamps REQ/FIL versions, updates verified_in on assessment subjects only
```

As-is graph today still allows orphans; treat that as a bug relative to the domain, not a feature. See Adding a domain layer.

Developer: relation sketch (as-is pair)

```
PR --HAS_REQUIREMENT--> REQ2026-001-042
PR --HAS_TEST_CASE----> TCAS2026-001-023
REQ --IS_REQUIREMENT_FOR_TEST_CASE--> TCAS
TCAS --IS_TEST_CASE_FOR_REQUIREMENT--> REQ
TCAS --IS_TEST_CASE_FOR_EXECUTION---> TCEX
TCEX --(evidence role)---------------> FIL2026-001-099
```

7. Scenario D – Document review (File is assessment subject)

Business story

SRS must be approved in FAT, not only uploaded.

1. Create a document-review verification item (or review workflow once Epic 4 exists) with assessment subject = `FIL2026-001-010`.
2. Reviewer passes CCC → File `verified_in` gains FAT.
3. Optional process/deliverable requirement may be linked; system-requirement coverage metrics ignore process REQs (when kind exists).

Contrast with Scenario B

Action	File present?	File verified_in
Link into dossier slot	yes	no
Pass document-review VI	yes	FAT added
Attach file as test evidence	n/a	no

```
Profile presence     --> readiness (P5) only
Document-review VI   --> File verified_in
System REQ + test VI --> REQ verified_in
Evidence / supporting input --> never verified_in from that link
```

---

8. Scenario E – Fail, re-run, then go / no-go

Fail then re-pass

1. First execution fails → REQ loses FAT from `verified_in` (if that was the last valid pass); `next required` = FAT.
2. Re-run passes → latest valid result wins → REQ `verified_in` includes FAT again.

Gate

1. Evaluate readiness — reads dossier presence, system-REQ verification, mandatory deliverable approvals, open P4 / blockers. Does not write subject `verified_in`.
2. Critical blocker open → go forbidden (transition guard fails).
3. No-go → project FSM stays in FAT; nothing on PhaseContext changes.
4. Go → project FSM transitions FAT → SAT (`isActive` moves); Object follower (if any) updates; items already verified in FAT follow; leftovers may lag on FAT; test home phase stays FAT.

Business view of go

Item	After go FAT → SAT
Project	Active ProjectPhase = SAT
Object (follower)	current = SAT; verified_in includes FAT
REQ verified in FAT	current = SAT; history kept
REQ not verified (non-critical leftover)	may stay current = FAT
Passed test case	home phase stays FAT
Open escalation finding	often stays P4

Resolving P on verify: always the project active phase (or the check's home phase taken from it at create). Do not look up Object to decide P.

9. Role playbook (business)

Role	Typical actions	Look at
Project manager	Enable phases, start phase FSM, request go/no-go	Project active phase, readiness, blockers
Dossier manager	Fill profile slots, run completeness	Missing/present slots, not verified_in
V&V engineer	Create checks, execute, raise findings	Coverage, pass/fail, evidence
Reviewer / CCC	Approve deliverables as assessment subjects	File verified_in after review only

UI must never offer free-form editors for `current` / `verified_in` / `next required` / `protocol`. Show them as read-only context (e.g. `FAT · P2`). Show the project active phase as the FSM state.

10. Developer cookbook

Always

1. Treat `ProjectPhase.isActive` as the only authority for the current phase.
2. Mutate PhaseContext only inside domain operations listed in the backlog.
3. Stamp versions of REQ/FIL on the execution, not on PhaseContext.
4. On pass: `verified_in += P` where P is the project active phase / check home phase.

Never

1. Let the client set `verified_in` after upload or completeness.
2. Create InputFile / OutputFile node types — use link roles.
3. Treat evidence attachment as approval of that File.
4. Put a second phase FSM on Object, or resolve P via Object↔Requirement for Epic 1.
5. Put PhaseContext on Structure, Contact, Order, … (Project has no PhaseContext node either — the FSM is `ProjectPhase`).
6. Switch the whole app into a “P5 mode” — protocol is a badge/filter, not a mode or FSM.

Mapping UI action → operation (short list)

UI	Must write	Must not write
Upload + link to slot	`HAS_LINK`	File `verified_in`
Completeness check	readiness flags, maybe blockers, protocol P5	File `verified_in`
Create requirement	REQ + PhaseContext init (`current`=project active P)	—
Pass test / VI	execution + subject `verified_in` += P	evidence File `verified_in`
Go	`ProjectPhase.isActive` → P′; follower PhaseContexts	rewrite historical executions

Full table: domain-model backlog.

As-is vs to-be (honest)

Topic	As-is today	To-be
Phase authority	Implicit / not modelled	Project phase FSM (`ProjectPhase.isActive`)
Object	Product node, status Finaal / Niet Finaal	Optional PhaseContext follower of the project FSM
Dossier	file Structure + `isComplete()`	+ slot overlay (obligation, phase, documentType), versioned profile
Check unit	TestCase / TestCaseExecution	VerificationItem (kinds beyond test)
Domain invariants	Enforced in FE / not at all	Enforced in domain operations / aggregates